

TIME SERIES FORECASTING AND TEXT ANALYSIS USING NLP

PROJECT TITLE : RETAIL SALES TREND & SEASONALITY ANALYSIS

SUBMITTED BY:

NAME : PRIYANKA B

REG NO : 727723EUA1093

DEPT & SEC : AI&DS - B

RETAIL SALES TREND & SEASONALITY ANALYSIS

1. Introduction

Retail businesses generate large volumes of sales data over time. Understanding how sales change across weeks, seasons, and special events is essential for making informed business decisions. Time series analysis helps identify long-term trends, recurring seasonal patterns, and random fluctuations in sales data. This project focuses on performing a comprehensive statistical analysis of retail sales data using the Walmart dataset to understand its underlying structure and suitability for future forecasting.

2. Problem Statement

A nationwide retail company operates multiple outlets and records sales transactions over time. Sales fluctuate due to factors such as seasonal demand, holidays, and promotional campaigns. Management lacks clarity on:

- Whether sales show a long-term upward or downward trend
- The impact of recurring seasonal patterns such as holidays
- The amount of randomness or noise present in the data
- Whether the sales data is stationary and suitable for forecasting models

Without understanding these characteristics, future predictive models may produce unreliable results, leading to poor business decisions.

3. Project Objectives

The main objectives of this project are:

- To analyze overall and store-wise retail sales trends
- To identify and quantify seasonal patterns in sales data
- To decompose the time series into trend, seasonality, and residual components
- To analyze autocorrelation and partial autocorrelation using ACF and PACF plots
- To test stationarity of the dataset using statistical methods
- To prepare the dataset for future forecasting models

4. Dataset Description

The project uses the **Walmart Sales Dataset** obtained from Kaggle. The dataset contains weekly sales data for multiple Walmart stores.

Key Attributes:

- **Store:** Unique store identifier
- **Date:** Week of sales record
- **Weekly_Sales:** Total weekly sales for the store
- **IsHoliday:** Indicates whether the week includes a major holiday
- **Temperature, Fuel_Price, CPI, Unemployment:** Additional economic and environmental factors

The dataset spans multiple years, making it suitable for trend and seasonality analysis.

5. System Requirements

- Python 3.x
- Pandas, NumPy
- Statsmodels
- Matplotlib, Seaborn

6. Methodology

6.1 Data Preprocessing

- Loaded the dataset using Pandas
- Converted the Date column to datetime format
- Sorted data chronologically
- Aggregated weekly sales across all stores for overall analysis

6.2 Exploratory Data Analysis

- Visualized weekly sales to observe trends and fluctuations
- Identified potential seasonal spikes around holidays

6.3 Time Series Decomposition

The sales time series was decomposed into:

- **Trend:** Long-term movement in sales
- **Seasonality:** Recurring patterns observed annually
- **Residuals:** Random noise and unexplained variations

An additive decomposition model with a 52-week period was used to reflect yearly seasonality.

6.4 Autocorrelation Analysis

- **ACF (Autocorrelation Function)** was used to analyze correlation between current and past sales values
- **PACF (Partial Autocorrelation Function)** helped identify direct relationships at specific lags

These plots helped confirm the presence of seasonality and non-stationarity.

6.5 Stationarity Testing

- The Augmented Dickey-Fuller (ADF) test was applied to check stationarity
- Since the series was non-stationary, first-order differencing was applied
- Stationarity was confirmed after differencing

7. Results and Observations

- The sales data shows a clear long-term trend
- Strong seasonal patterns were observed, especially during holiday periods
- Significant autocorrelation indicated dependence on previous weeks' sales
- The original series was non-stationary but became stationary after differencing

8. Conclusion

This project successfully analyzed the Walmart retail sales data using time series statistical techniques. The analysis revealed meaningful trends and seasonal patterns, validated the presence of autocorrelation, and ensured stationarity for future forecasting. The prepared dataset is now suitable for building predictive models such as ARIMA or SARIMA.

9. Future Scope

- Build forecasting models (ARIMA/SARIMA)
- Perform store-level comparative forecasting
- Analyze the impact of economic indicators on sales
- Extend analysis to monthly or quarterly sales aggregation

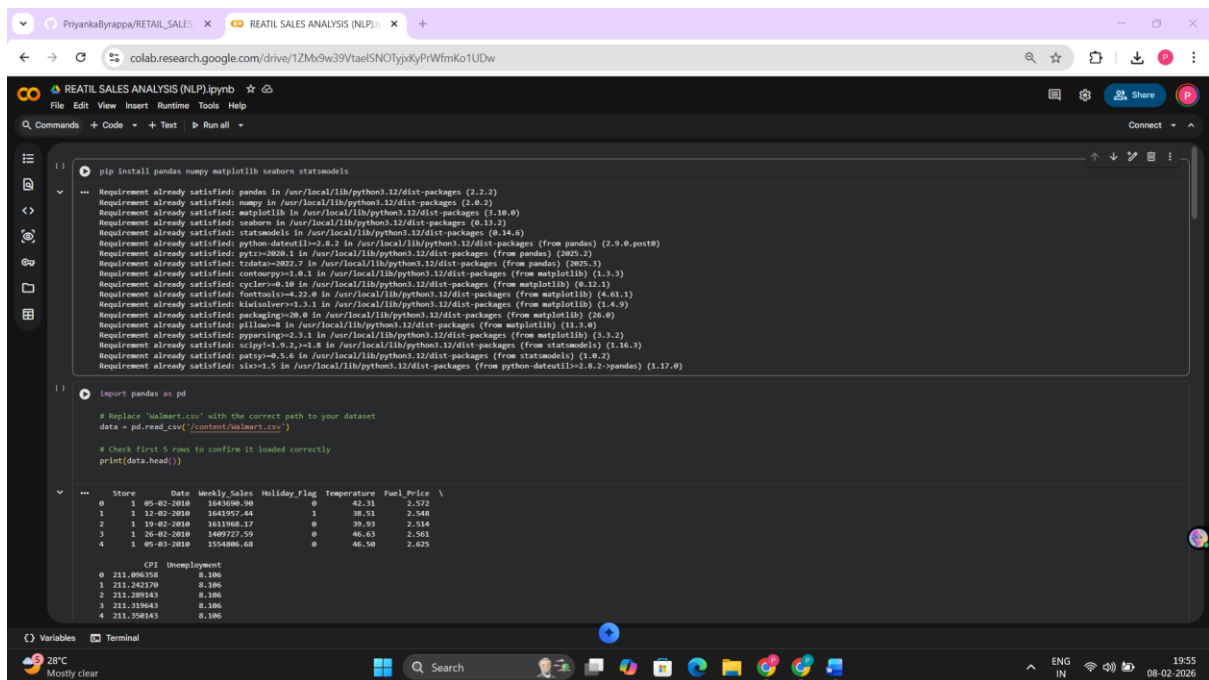
10. Tools and Technologies Used

- Python
- Pandas and NumPy for data handling
- Statsmodels for time series analysis
- Matplotlib and Seaborn for visualization

11. References

- Walmart Sales Dataset – Kaggle
- Statsmodels Documentation
- Time Series Analysis Concepts

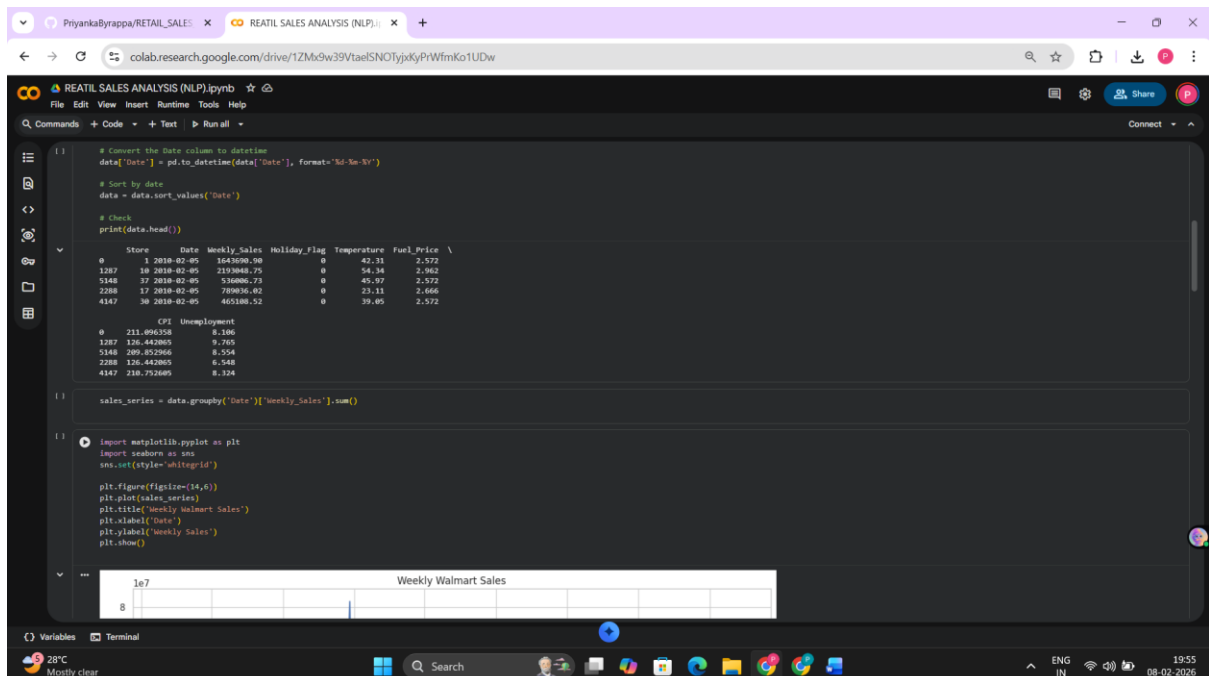
SCREENSHOTS (OUTPUT):



The screenshot shows a Google Colab notebook titled "RETAIL SALES ANALYSIS (NLP).ipynb". The first cell contains the command `pip install pandas numpy matplotlib seaborn statsmodels`, followed by a list of requirements that are already satisfied. The second cell imports `pandas` as `pd` and loads a dataset from a local file path. The output of the second cell shows the first 5 rows of the dataset, which includes columns for Store, Date, Weekly_Sales, Holiday_Flag, Temperature, and Fuel_Price. Below the first 5 rows, the output shows the first 5 rows of the dataset, which includes columns for Store, Date, Weekly_Sales, Holiday_Flag, Temperature, and Fuel_Price.

```
1 | pip install pandas numpy matplotlib seaborn statsmodels
  | Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
  | Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
  | Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
  | Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
  | Requirement already satisfied: statsmodels in /usr/local/lib/python3.12/dist-packages (0.14.0)
  | Requirement already satisfied: python-dateutil<2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
  | Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
  | Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
  | Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
  | Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
  | Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
  | Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
  | Requirement already satisfied: packaging>=20.8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
  | Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.1.0)
  | Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
  | Requirement already satisfied: scipy<1.9.2,>=1.8 in /usr/local/lib/python3.12/dist-packages (from statsmodels) (1.16.3)
  | Requirement already satisfied: patsy>=0.5.4 in /usr/local/lib/python3.12/dist-packages (from statsmodels) (1.8.2)
  | Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2;pandas) (1.17.0)

2 | import pandas as pd
  | # Replace 'Walmart.csv' with the correct path to your dataset
  | data = pd.read_csv('/content/Walmart.csv')
  | # Check first 5 rows to confirm it loaded correctly
  | print(data.head())
  |
  | Store    Date    Weekly_Sales  Holiday_Flag  Temperature  Fuel_Price \
  | 0  1 05-02-2010  164500.90          0         42.31      2.572
  | 1  1 12-02-2010  164197.44          1         38.51      2.548
  | 2  1 19-02-2010  161108.17          0         39.83      2.514
  | 3  1 26-02-2010  140727.50          0         46.63      2.561
  | 4  1 05-03-2010  155486.68          0         46.50      2.625
  |
  | CPI  Unemployment
  | 0  211.896358      8.106
  | 1  211.842170      8.106
  | 2  211.289143      8.106
  | 3  211.333043      8.106
  | 4  211.258143      8.106
```



The screenshot shows the same Google Colab notebook. The third cell converts the 'Date' column to a datetime format and sorts the data by date. The output shows the first 5 rows of the sorted dataset. The fourth cell groups the data by date and calculates the sum of weekly sales. The output shows the first 5 rows of the grouped data. The fifth cell imports `matplotlib.pyplot` as `plt` and `seaborn` as `sns`, and creates a line plot of weekly Walmart sales. The plot shows a line graph with 'Date' on the x-axis and 'Weekly Sales' on the y-axis. The y-axis has a multiplier of $1e7$. The plot is titled 'Weekly Walmart Sales'.

```
1 | # Convert the Date column to datetime
  | data['Date'] = pd.to_datetime(data['Date'], format='%d-%m-%Y')
  | # Sort by date
  | data = data.sort_values('Date')
  | # Check
  | print(data.head())
  |
  | Store    Date    Weekly_Sales  Holiday_Flag  Temperature  Fuel_Price \
  | 0  1 2010-02-05  164500.90          0         42.31      2.572
  | 1287 18 2010-02-05  213388.75          0         54.34      2.962
  | 5148 37 2010-02-05  516006.73          0         45.97      2.572
  | 2288 17 2010-02-05  70936.02          0         23.11      2.666
  | 4147 36 2010-02-05  463180.52          0         39.45      2.572
  |
  | CPI  Unemployment
  | 0  211.896358      8.106
  | 1287 126.442805      9.765
  | 5148 209.832946      8.544
  | 2288 126.442805      8.548
  | 4147 218.752685      8.124

2 | sales_series = data.groupby('Date')['Weekly_Sales'].sum()

3 | import matplotlib.pyplot as plt
  | import seaborn as sns
  | sns.set(style='whitegrid')
  |
  | plt.figure(figsize=(14,6))
  | plt.plot(sales_series)
  | plt.title('Weekly Walmart Sales')
  | plt.xlabel('Date')
  | plt.ylabel('Weekly Sales')
  | plt.show()
  |
  | 1e7
  | 8
  | Weekly Walmart Sales
```

